

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de TADs**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de **Entre Ríos**

PROBLEMA N° 2 GUERRA DE CARTAS

2. INTRODUCCIÓN

Este trabajo aborda la implementación de un **TAD (Tipo Abstracto de Datos)** denominado "**Mazo**", diseñado para simular un juego de cartas llamado "**Guerra de cartas**". El problema central consiste en representar un **mazo** que soporte operaciones eficientes de inserción y extracción en ambos extremos. Para resolverlo se aplicaron los conceptos de lista doblemente enlazada (**LDE**) y manejo de excepciones personalizadas en Python. Los docentes proporcionaron la clase **Carta** y la lógica del juego (**JuegoGuerra**); la tarea consistió en implementar la clase **Mazo** de forma que el juego y los tests provistos funcionaran correctamente. El informe va a describir el análisis del problema, las decisiones de implementación y los resultados obtenidos.

3. DESARROLLO

A. Análisis y planteo

El problema requiere un mazo con tres operaciones críticas: agregar cartas en la parte superior (**poner_carta_arriba**), agregar en la parte inferior (**poner_carta_abajo**) y extraer desde la parte superior (**sacar_carta_arriba**). En Python hay funciones de lista cubren estos casos, pero **list.insert()** tiene complejidad **O(n)** porque desplaza todos los elementos. En cambio, la **LDE** permite insertar y extraer desde ambos extremos en **O(1)**, ya que basta con redirigir los punteros de cabeza o cola. Por eso el **TAD Mazo** usa internamente la **LDE** implementada en el **Problema_1** como estructura de almacenamiento. Esta elección es razonable porque el juego opera constantemente sobre los extremos del **mazo** (sacar de arriba, poner abajo al ganar un turno), haciendo que la diferencia entre **O(1)** y **O(n)** sea significativa a lo largo de partidas de cientos o miles de turnos.

B. Implementación / Resolución

La clase **DequeEmptyError** está destinada a manejar el error que se produce al intentar extraer una carta de un mazo vacío. Evita fallos inesperados y facilita la depuración, haciendo el programa más seguro.



Cada elemento del mazo se representa mediante un nodo doblemente enlazado. De esta manera se construye una lista doble que permite recorrer y modificar el mazo con eficiencia. La clase **Mazo** gestiona la baraja de cartas.

Sus principales funciones son:

- **Inicialización (`__init__`):** Crea un mazo vacío. Que internamente usa una **ListaDobleEnlazada**, lo que permite insertar y extraer desde ambos extremos en **O(1)**.
- **Poner la carta arriba (`poner_carta_arriba`):** Delega en **agregar_al_inicio** de la **LDE**: Crea un nuevo nodo que enlaza antes de la cabeza actual y actualiza la referencia de cabeza. En complejidad **O(1)**.
- **Poner la carta abajo (`poner_carta_abajo`):** Delega en **agregar_al_final**: Crea un nodo, luego lo enlaza después del nodo cola y actualiza la referencia de cola. También **O(1)**.
- **Sacar la carta de arriba (`sacar_carta_arriba`):** Tiene tres responsabilidades:
 1. Verifica si el mazo está vacío antes de operar.
 2. Extrae la carta en el **índice0** usando **extraer(0)** de la **LDE**
 3. Si **mostrar = True**, marca la carta como visible antes de devolverla. Si el mazo está vacío, lanza **DequeEmptyError**.
- **Consultar el tamaño de cartas (`__len__`):** Devuelve el número de cartas contenidas en el mazo en **O(1)**, delegando en **len(self._cartas)**.
- **Representación en texto (`__str__`):** Permite usar **print(mazo)** y ver algo como {As♠, 7♥, K♦}, útil para depurar o mostrar el estado del juego.
- **Verificar si está vacío (`esta_vacio`):** Retorna un valor booleano que indica si el mazo tiene o no cartas. Se usa dentro de **sacar_carta_arriba** como guardia antes de operar.



Tabla de Pre y Postcondiciones:

Método	Precondición	Postcondición
poner_carta_arriba	Objeto Carta válido.	La carta se inserta en la posición0 ; el tamaño de la lista se incrementa en 1 .
sacar_carta_arriba	El mazo no debe estar vacío (longitud > 0).	Retorna y elimina la carta superior. Si está vacío, lanza DequeEmptyError .
poner_carta_abajo	Objeto Carta válido.	La carta se agrega al final de la estructura en tiempo constante O(1) .
__len__	Ninguna.	Retorna la cantidad actual de cartas en el mazo en tiempo O(1) .

Excepciones y Contratos

- ❖ **__init__**, Inicialización del mazo:

```
def __init__(self):  
    self._cartas = ListaDobleEnlazada()
```

- ❖ **poner_carta_arriba** y **poner carta abajo**:

```
def poner_carta_arriba(self, carta):  
    self._cartas.agregar_al_inicio(carta)  
  
def poner_carta_abajo(self, carta):  
    self._cartas.agregar_al_final(carta)
```



- ❖ **agregar_al_inicio:** La carta queda en la cima del mazo (posición 0):

```
def agregar_al_inicio(self, item):
    nuevo = Nodo(item)
    if self.esta_vacia():
        self.__cabeza = self.__cola = nuevo
    else:
        nuevo.asignar_siguiente(self.__cabeza)
        self.__cabeza.asignar_anterior(nuevo)
        self.__cabeza = nuevo
    self.__tamano += 1
```

- ❖ **agregar_al_final:** La carta queda en el fondo del mazo:

```
def agregar_al_final(self, item):
    nuevo = Nodo(item)
    if self.esta_vacia():
        self.__cabeza = self.__cola = nuevo
    else:
        self.__cola.asignar_siguiente(nuevo)
        nuevo.asignar_anterior(self.__cola)
        self.__cola = nuevo
    self.__tamano += 1
```

Esto modela perfectamente un mazo real donde podés poner cartas encima o abajo.

- ❖ **sacar_carta_arriba:**

```
def sacar_carta_arriba(self, mostrar=False):
    if self._esta_vacio():
        raise DequeEmptyError("El mazo está vacío")
    carta = self._cartas.extraer(0) # ← usamos extraer directamente
    if mostrar:
        carta.visible = True
    return carta
```



❖ `_esta_vacio`:

```
def _esta_vacio(self):  
    return len(self._cartas) == 0
```

Simplemente consulta si la longitud de la lista es cero. Se usa dentro de **sacar_carta_arriba** como guardia antes de operar..

❖ `__len__` y `__str__`:

```
def __len__(self):  
    return len(self._cartas)  
  
def __str__(self):  
    return "{" + ",".join([str(carta) for carta in self._cartas]) + "}"
```

`__len__`: Permite usar **len(mazo)** directamente, como si fuera una lista nativa de Python.

`__str__`: Permite usar **print(mazo)** y ver algo como {As♠, 7♥, K♦}, lo que es útil para depurar o mostrar el estado del juego.

Excepciones y Contratos:

Para garantizar que la simulación del juego sea estable y maneje correctamente el fin de la partida, se implementaron los siguientes mecanismos de control:

Manejo de Excepciones: Se definió e implementó la clase **DequeEmptyError**. Esta excepción es fundamental para la lógica del simulador, ya que es lanzada por **sacar_carta_arriba()** cuando se intenta extraer de un mazo vacío, permitiendo una finalización controlada del juego.



C. Resultados y pruebas

A continuación se muestran los resultados de ejecutar los tests provistos por la cátedra. Los tests validan tanto el comportamiento de la clase **Mazo** como la ejecución completa del juego **Guerra de Cartas**.

Tests de la clase Mazo (test_mazo.py):

```
test_poner_abajo (tests.test_mazo.TestMazo.test_poner_abajo) ... ok
test_poner_sacar_arriba (tests.test_mazo.TestMazo.test_poner_sacar_arriba) ... ok

-----
Ran 2 tests in 0.001s

OK
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_2> |
```

Tests del juego Guerra (test_juegoguerra.py):

```
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_2> cd c:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_2
>> python -m unittest tests.test_juegoguerra -v
test_resulta_empate (tests.test_juegoguerra.TestJuegoGuerra.test_resulta_empate)
compruebo el resultado de 2 partidas con empate ... ok
test_resulta_gana_jugador1 (tests.test_juegoguerra.TestJuegoGuerra.test_resulta_gana_jugador1)
compruebo el número de turnos de 3 partidas con ... ok
test_resulta_gana_jugador2 (tests.test_juegoguerra.TestJuegoGuerra.test_resulta_gana_jugador2)
compruebo el número de turnos de 3 partidas con ... ok

-----
Ran 3 tests in 0.169s

OK
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_2> |
```

4. ANÁLISIS DE RESULTADOS ESPECÍFICOS / DISCUSIÓN

La implementación demuestra que el uso de un **TAD** basado en una **Lista Doblemente Enlazada** es superior para este caso de uso específico. A continuación se analizan los resultados obtenidos:

Eficiencia de operaciones: Las operaciones **poner_carta_arriba** y **poner_carta_abajo** se ejecutan en **O(1)** en todos los casos, a diferencia de una lista Python donde **insert(0, x)** sería **O(n)**. En una partida típica de **200 - 600 turnos**, cada turno realiza al menos **4** operaciones sobre los mazos, lo que significa miles de **inserciones/extracciones** que se benefician directamente de la complejidad constante de la **LDE**.

Manejo de fin de partida: La excepción **DequeEmptyError** permite que **guerra_de_cartas.py** detecte el fin de partida de forma controlada: Cuando un jugador se queda sin cartas durante un turno de guerra (situación especial donde se apuestan 3 cartas adicionales), el bloque **except DequeEmptyError** declara ganador al otro jugador sin producir un crash.

Validación por tests: Los **2** tests provistos por la cátedra, pasaron sin modificaciones al



Universidad Nacional
de Entre Ríos

código provisto. Los resultados de turnos jugados coinciden exactamente con los valores esperados con el límite de **1000** turnos, validando la correcta implementación de la lógica del mazo.

Comparación con alternativas: Usar una **deque** de Python (**collections.deque**) habría dado el mismo rendimiento **O(1)**, pero el objetivo del trabajo era implementar el **TAD** desde cero usando la **LDE** del **Problema_1**, demostrando comprensión de las estructuras subyacentes.

5. DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Durante la elaboración de este trabajo se utilizaron herramientas de inteligencia artificial generativa como apoyo en algunas etapas del proceso.

Herramientas utilizadas:

- Claude AI

Descripción del uso:

- ➔ **Consulta teórica:** Se utilizó para clarificar conceptos sobre listas doblemente enlazadas, deque y manejo de excepciones en Python, complementando la bibliografía de la materia.
- ➔ **Asistencia en redacción:** Se usó para mejorar la redacción de algunas secciones del informe, como la introducción y el desarrollo, asegurando claridad y coherencia.
- ➔ **Asistencia en código:** Se consultó para depurar errores de sintaxis en la implementación de la clase **Mazo** y verificar la correcta gestión de excepciones.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de **IA**. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.



Universidad Nacional
de Entre Ríos

6. TRABAJO EN EQUIPO

Organización y roles:

Franco Acevedo se encargó principalmente del diseño e implementación de la clase **Mazo** y sus métodos principales (`__init__`, `poner_carta_arriba`, `poner_carta_abajo`, `sacar_carta_arriba`).

Franco Tomiozzo trabajó en la implementación de los métodos auxiliares (`esta_vacio`, `__len__`, `__str__`), la clase **DequeEmptyError** y la redacción del informe.

Coordinación y comunicación:

La coordinación se realizó principalmente mediante WhatsApp para la comunicación diaria y el repositorio de GitHub compartido para la integración del código. Cuando surgieron diferencias sobre cómo implementar la lista doblemente enlazada interna, se resolvió mediante una reunión virtual por videollamada donde se analizaron las opciones y se decidió utilizar la estructura ya provista por la cátedra.

Estimación y tiempo real:

La estimación inicial fue de aproximadamente **6 horas** en total: **1 hora** de lectura y análisis del enunciado, **3 horas** de desarrollo e implementación, **1 hora** de pruebas y **1 hora** de redacción del informe. El tiempo real insumido fue de aproximadamente **8 horas**. La diferencia se debió principalmente a dificultades en la comprensión del funcionamiento del **Juego de cartas** y en la integración de la clase **Mazo** con las clases provistas por la cátedra, lo que requirió mayor tiempo de prueba y corrección del que se había previsto.

7. CONCLUSIÓN

Los objetivos del trabajo se cumplieron: la clase **Mazo** implementa correctamente todas las operaciones requeridas y los tests provistos por la cátedra pasan sin errores. Usar una **Lista Doblemente Enlazada** ya que permite manejar las operaciones del mazo de manera eficiente desde ambos extremos en tiempo constante **O(1)**.

La principal dificultad fue comprender cómo interactúa el **Mazo** con la lógica de `guerra_de_cartas.py`, especialmente el manejo del estado de guerra y el uso de **DequeEmptyError** como señal de fin de turno, lo que requirió mayor tiempo de prueba del estimado. Esta integración evidenció la importancia de diseñar las excepciones



Universidad Nacional
de Entre Ríos

personalizadas como parte del contrato de la interfaz del **TAD**, no sólo como mecanismo de control interno.

Como aprendizaje central, quedó claro que elegir la estructura de datos correcta desde el análisis inicial impacta directamente en la eficiencia del sistema completo. Cada parte del programa funcionó de forma coherente y los tests del juego se ejecutaron sin inconvenientes, validando que la estructura diseñada resuelve el problema correctamente.

8. REFERENCIAS Y BIBLIOGRAFÍA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.

Documentación oficial de Python — collections.deque:

<https://docs.python.org/3/library/collections.html#collections.deque>

Material de cátedra: Fundamentos de Algoritmos y Estructuras de Datos, UNER Facultad de Ingeniería, 2026.

Repositorio del trabajo:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de **Entre Ríos**